# Stat 20 Summer 2026 — Review Session 1

**Summarizing Data, the R Toolkit, and Probability Foundations**

Curtis McDonald

## Table of contents

**How to use these notes.** Each section states the *idea* first, then shows the *R code* you'd actually run. All examples use the `penguins` data from `stat20data`, so you can copy any chunk into RStudio and run it. Load the libraries once at the top of your session:

```
library(tidyverse)
library(stat20data)
library(broom)
data(penguins)
```

# 1 Types of Claims

Everything in this course is about **constructing and critiquing claims made with data**. There are four flavors:

| Claim | What it does | Example |
|---|---|---|
| **Summary** | Describes data *you have on hand* | "70% of survey respondents had never coded." |
| **Generalization** | Extends a sample to a *broader population* | "70% of *Berkeley students* have never coded." |
| **Causal** | Says changing one variable *changes* another | "The new antibiotic *eliminates* >99% of infections." |
| **Prediction** | Guesses an unknown value from known ones | "Uber stock will rise 1.2% tomorrow." |

Session 1 lives almost entirely in the **summary** world (plus the probability machinery that later powers generalization).

## 2 Types of Data & the Unit of Observation

A **variable** is a characteristic measured on each observation. The **taxonomy of data** splits every variable into a type:

- **Numerical** — magnitudes with quantitative meaning
  - *Continuous*: any value on an interval (e.g. `bill_length_mm`)
  - *Discrete*: values with jumps (e.g. household size)
- **Categorical** — values are categories (each unique one is a *level*)
  - *Ordinal*: levels have a natural order (e.g. "small < medium < large")
  - *Nominal*: no order (e.g. `species`, `island`)

The **unit of observation** is the *kind of thing* each row represents. In `penguins`, one row = one penguin.

```
glimpse(penguins)
```

```
Rows: 333
Columns: 8
$ species           <fct> Adelie, Adelie, Adelie, Adelie, Adelie, Adelie, Adel~
$ island            <fct> Torgersen, Torgersen, Torgersen, Torgersen, Torgerse~
$ bill_length_mm    <dbl> 39.1, 39.5, 40.3, 36.7, 39.3, 38.9, 39.2, 41.1, 38.6~
$ bill_depth_mm     <dbl> 18.7, 17.4, 18.0, 19.3, 20.6, 17.8, 19.6, 17.6, 21.2~
$ flipper_length_mm <int> 181, 186, 195, 193, 190, 181, 195, 182, 191, 198, 18~
$ body_mass_g       <int> 3750, 3800, 3250, 3450, 3650, 3625, 4675, 3200, 3800~
$ sex               <fct> male, female, female, female, male, female, male, fe~
$ year              <int> 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007~
```

> Exam move
>
> To classify a variable: (1) numbers or categories? (2) if numbers, continuous or discrete? if categories, ordered or not? A logical (`TRUE`/`FALSE`) variable behaves like a nominal categorical *and* like a 0/1 number.

## 3 Distributions: Shape

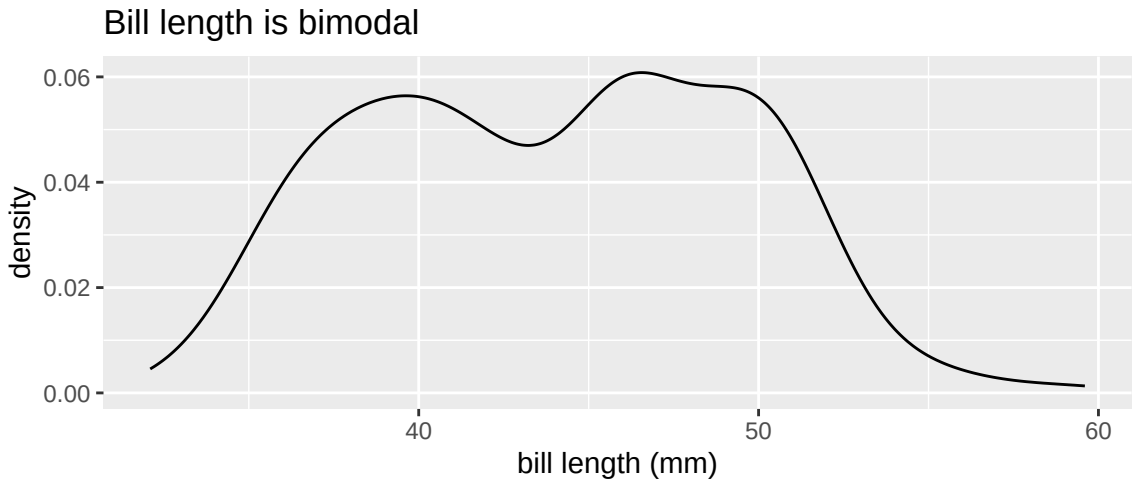
When we describe a distribution we talk about its **shape, center, and spread**.

**Shape** has two main descriptors:

- **Modality** — number of distinct peaks: *unimodal* (one), *bimodal* (two), ...
- **Skew** — which tail stretches out:
  - *Right (positive) skew*: long right tail (e.g. US household income)
  - *Left (negative) skew*: long left tail
  - *Symmetric*: both tails similar

A **uniform** distribution is flat — every value (or interval) is equally likely. A fair die roll is the classic discrete uniform.

```
# bill length across all penguins is bimodal (small Adelies vs larger species)
ggplot(penguins, aes(x = bill_length_mm)) +
  geom_density() +
  labs(title = "Bill length is bimodal", x = "bill length (mm)")
```




# 4 Measures of Center & Spread

## 4.1 Center: mean vs. median

$$\bar{x} = \frac{x_1 + x_2 + \cdots + x_n}{n} \qquad \text{median} = \text{middle value (sorted)}$$

- The **mean** uses every value's magnitude, so it is **pulled by outliers**.
- The **median** only uses order, so it is **resistant (robust)** to outliers.

Rule of thumb: for a **skewed** distribution the median is the more honest "typical" value (this is why we report *median* household income).

### 4.2 Spread: variance, SD, IQR

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2 \qquad s = \sqrt{s^2} \qquad IQR = Q_3 - Q_1$$

- **Variance** $s^2$: average squared distance from the mean (units are squared).
- **Standard deviation** $s$: square root of variance — same units as the data.
- **IQR**: width of the middle 50%. Like the median, it is **robust** to outliers.

```
penguins |>
  summarize(
    mean_mass   = mean(body_mass_g, na.rm = TRUE),
    median_mass = median(body_mass_g, na.rm = TRUE),
    sd_mass     = sd(body_mass_g, na.rm = TRUE),
    iqr_mass    = IQR(body_mass_g, na.rm = TRUE)
  )
```

```
# A tibble: 1 x 4
  mean_mass median_mass sd_mass iqr_mass
      <dbl>       <int>   <dbl>    <dbl>
1     4207.        4050    805.     1225
```

## 5 Contingency Tables & Proportions

A **contingency table** counts observations in every combination of two categorical variables.

```
table(penguins$species, penguins$island)
```

```
           Biscoe Dream Torgersen
  Adelie       44    55        47
  Chinstrap     0    68         0
  Gentoo      119     0         0
```

From counts we get three kinds of proportions. Let $A$ = species, $B$ = island:

- **Joint** $P(A \cap B)$: in a cell, out of the *whole* table. (Gentoo *and* Biscoe) = 119/344.
- **Marginal** $P(B)$: a whole row/column total, out of the whole table. (Biscoe) = 163/344.
- **Conditional** $P(A \mid B) = \dfrac{P(A \cap B)}{P(B)}$: within one level of the other variable. (Gentoo *given* Biscoe) = 119/163.

`prop.table()` turns counts into proportions; `margin = 2` conditions on the column variable:

```
# conditional distribution of species WITHIN each island (columns sum to 1)
round(prop.table(table(penguins$species, penguins$island), margin = 2), 2)
```

```
           Biscoe Dream Torgersen
  Adelie     0.27  0.45      1.00
  Chinstrap  0.00  0.55      0.00
  Gentoo     0.73  0.00      0.00
```

Two categorical variables are **associated** if the conditional proportions *change* as you move across levels of the conditioning variable.

## 6 Visualizing with `ggplot2` (aes + geom)

Every ggplot is built from three pieces:

1. **data** — the data frame,
2. **aesthetics** `aes()` — which *variable* maps to which visual channel (`x`, `y`, `color`, `fill`, ...),
3. **geometry** `geom_*()` — the shape used to draw each observation.

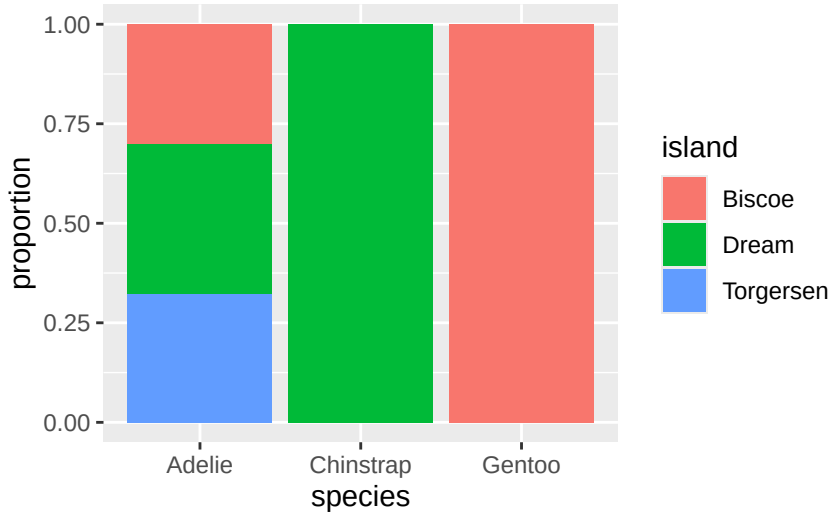
```
ggplot(data = DATAFRAME, mapping = aes(x = VAR, y = VAR)) +
  geom_SHAPE()
```

> **i** Mapping vs. setting
>
> Put an attribute **inside** `aes()` to tie it to a variable (`aes(color = species)`). Put it **outside** `aes()`, inside the geom, to set a fixed look (`geom_point(color = "blue")`).

### 6.1 Bar chart — one or two categorical variables

`position = "fill"` gives a normalized (conditional-proportion) bar chart; `"dodge"` gives side-by-side bars; the default is stacked.

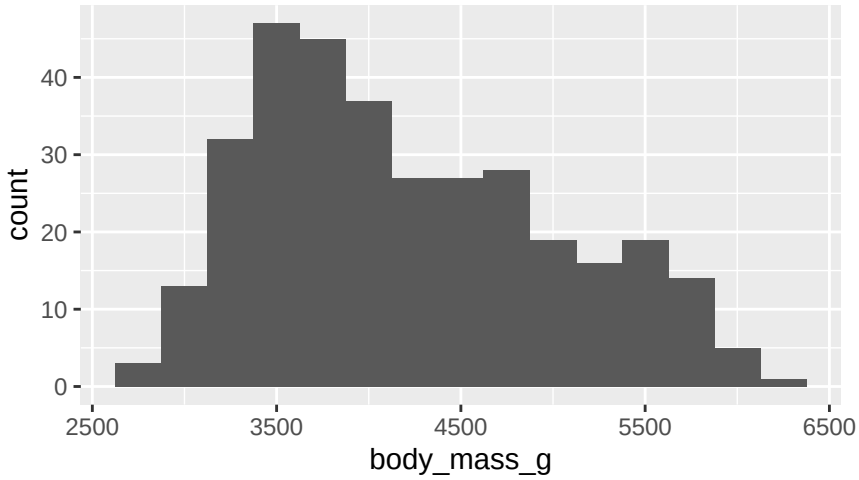
```
ggplot(penguins, aes(x = species, fill = island)) +
  geom_bar(position = "fill") +
  labs(y = "proportion")
```




## 6.2 Histogram — one numerical variable

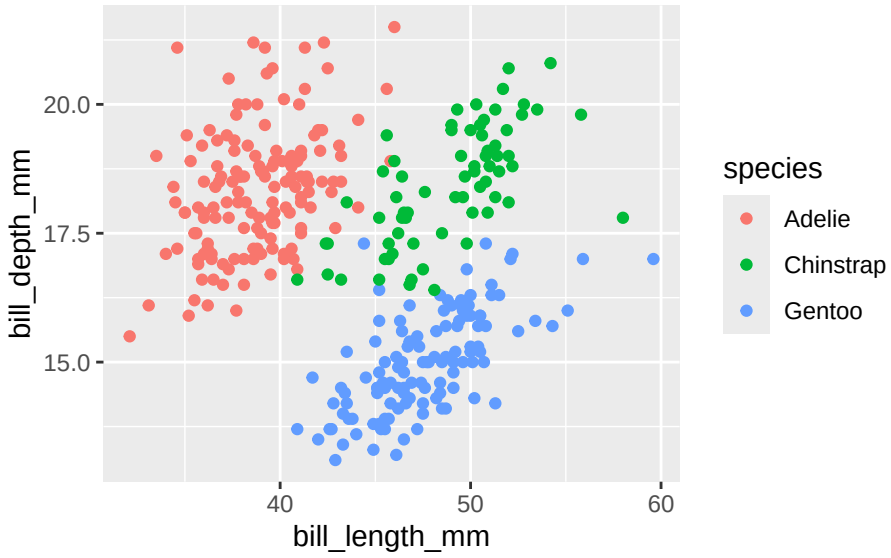
Bins the range, then counts. Tune `binwidth` (or `bins`) to see more/less detail.

```
ggplot(penguins, aes(x = body_mass_g)) +
  geom_histogram(binwidth = 250)
```




## 6.3 Scatter plot — two numerical variables

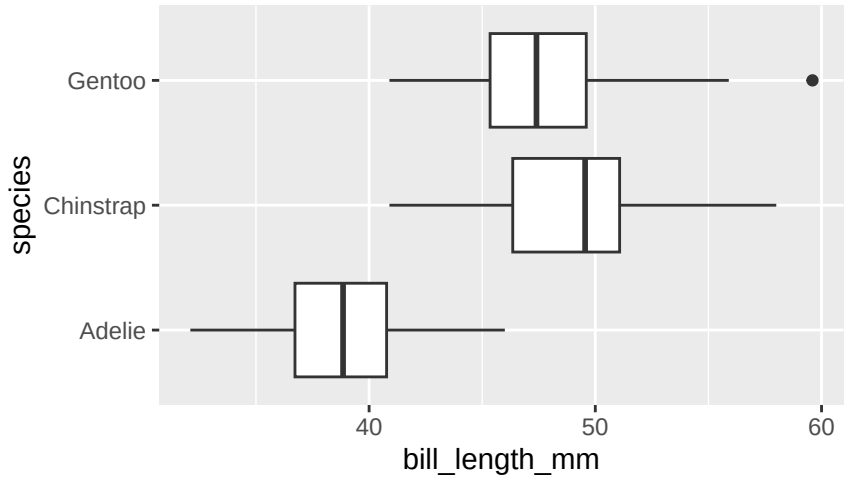
```
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm, color = species)) +
  geom_point()
```




## 6.4 Box plot — numerical distribution across groups

The box spans $Q_1$ to $Q_3$ (its width is the IQR); the line inside is the median; whiskers reach $1.5 \cdot \text{IQR}$; dots beyond are outliers.

```
ggplot(penguins, aes(x = bill_length_mm, y = species)) +
  geom_boxplot()
```



## 7 Wrangling with `dplyr`

`dplyr`'s verbs each take a data frame as the first argument and return a data frame, so you chain them with the **pipe** `|>` ("then"). Read `df |> f() |> g()` as "take `df`, then `f`, then `g`."

| Verb | What it does |
|---|---|
| `filter()` | keep **rows** matching a condition |
| `select()` | keep (or drop with `-`) **columns** |
| `arrange()` | sort rows (`desc()` for descending) |
| `mutate()` | create / modify a **column** |
| `summarize()` | collapse to summary statistics (one row) |
| `group_by()` | do the next operation **per group** |

**Comparison operators:** `==`, `!=`, `<`, `>`, `<=`, `>=`. **Logical operators:** `&` (and), `|` (or), `%in%` (in a set).

```
# filter + select + arrange
penguins |>
  filter(species == "Gentoo", body_mass_g > 5500) |>
  select(species, island, body_mass_g) |>
  arrange(desc(body_mass_g)) |>
  head(4)
```

```
# A tibble: 4 x 3
  species island body_mass_g
  <fct>   <fct>        <int>
1 Gentoo  Biscoe        6300
2 Gentoo  Biscoe        6050
3 Gentoo  Biscoe        6000
4 Gentoo  Biscoe        6000
```

```
# mutate: add a new column
penguins |>
  mutate(bill_ratio = bill_length_mm / bill_depth_mm) |>
  select(species, bill_length_mm, bill_depth_mm, bill_ratio) |>
  head(3)
```

```
# A tibble: 3 x 4
  species bill_length_mm bill_depth_mm bill_ratio
```

```
  <fct>            <dbl>         <dbl>      <dbl>
1 Adelie            39.1          18.7       2.09
2 Adelie            39.5          17.4       2.27
3 Adelie            40.3          18         2.24
```

```r
# group_by + summarize: one summary row PER group
penguins |>
  group_by(species) |>
  summarize(
    mean_mass = mean(body_mass_g, na.rm = TRUE),
    n = n()
  )
```

```
# A tibble: 3 x 3
  species   mean_mass     n
  <fct>         <dbl> <int>
1 Adelie        3706.   146
2 Chinstrap     3733.    68
3 Gentoo        5092.   119
```

> The 0/1 trick
>
> A comparison like `body_mass_g > 5000` is a logical vector of `TRUE`/`FALSE`, i.e. 1s and 0s. So `mean(body_mass_g > 5000)` gives the **proportion** above 5000.

# 8 Correlation & Linear Models

## 8.1 Correlation coefficient $r$

$r$ measures the **strength and direction of a linear** relationship; it is always between $-1$ and 1 and has no units.

```r
penguins |>
  summarize(r = cor(bill_length_mm, bill_depth_mm, use = "complete.obs"))
```

```
# A tibble: 1 x 1
      r
   <dbl>
1 -0.229
```

(Here $r$ is negative overall — a *Simpson's paradox* teaser: within each species the relationship is positive.)

## 8.2 Fitting a line with `lm()`

The **least squares line** $\hat{y} = b_0 + b_1 x$ is the line minimizing the residual sum of squares. Fit it with `lm(y ~ x, data = ...)`:

```
m1 <- lm(body_mass_g ~ flipper_length_mm, data = penguins_c)
m1              # prints the intercept (b0) and slope (b1)
```

```

Call:
lm(formula = body_mass_g ~ flipper_length_mm, data = penguins_c)

Coefficients:
      (Intercept)  flipper_length_mm
         -5872.09              50.15
```

Read the output as

$$\widehat{\text{body\_mass}} = b_0 + b_1 \cdot \text{flipper\_length}.$$

The slope $b_1$ is the expected change in $y$ for a **one-unit** increase in $x$.

**Coefficients on their own** (base R or `broom::tidy()`):

```
coef(m1)                    # named vector of b0, b1
```

```
      (Intercept) flipper_length_mm
      -5872.09268          50.15327
```

```
tidy(m1) |> select(term, estimate, std.error)
```

```
# A tibble: 2 x 3
  term              estimate std.error
  <chr>                <dbl>     <dbl>
1 (Intercept)        -5872.     310.
2 flipper_length_mm     50.2      1.54
```

## 8.3 Model quality with `glance()`

`broom::glance()` reports one-row model summaries; `r.squared` is the share of variability in $y$ explained by the model ($0$ = no better than $\bar{y}$, $1$ = perfect):

```
glance(m1) |> select(r.squared)
```

```
# A tibble: 1 x 1
  r.squared
      <dbl>
1     0.762
```

## 8.4 Residuals

A **residual** is observed minus predicted, $\hat{e}_i = y_i - \hat{y}_i$. Get fitted values and residuals and glue them back on with `mutate()`:

```
penguins_c |>
  mutate(
    y_hat = fitted(m1),   # predicted body mass
    e_hat = resid(m1)     # residual = actual - predicted
  ) |>
  select(flipper_length_mm, body_mass_g, y_hat, e_hat) |>
  head(3)
```

```
# A tibble: 3 x 4
  flipper_length_mm body_mass_g y_hat e_hat
              <int>       <int> <dbl> <dbl>
1               181        3750 3206.  544.
2               186        3800 3456.  344.
3               195        3250 3908. -658.
```

## 8.5 Predicting with `predict()`

Build a small data frame of new predictor values and pass it as `newdata`:

```
new_birds <- data.frame(flipper_length_mm = c(190, 215))
predict(m1, newdata = new_birds)
```

```
       1        2
3657.028 4910.859
```

# 9 Multiple Linear Regression

Add predictors with `+`. With **two numerical** predictors you fit a plane; adding a predictor can only **raise** $R^2$ on the training data.

```
m2 <- lm(body_mass_g ~ flipper_length_mm + bill_length_mm, data = penguins_c)
tidy(m2) |> select(term, estimate)
```

```
# A tibble: 3 x 2
  term              estimate
  <chr>                <dbl>
1 (Intercept)       -5836.
2 flipper_length_mm    48.9
3 bill_length_mm        4.96
```

## 9.1 Categorical predictor = a "shift" (dummy variable)

A categorical predictor with $k$ levels becomes $k-1$ **dummy (0/1) variables**. The left-out level is the **reference**; each dummy's coefficient is the vertical **shift** from that reference. Here, using Adelie penguins, `sex` shifts the line:

```
adelie <- penguins_c |> filter(species == "Adelie")
m3 <- lm(body_mass_g ~ sex + flipper_length_mm, data = adelie)
coef(m3)
```

```
      (Intercept)           sexmale flipper_length_mm
        305.08667         599.34326          16.31437
```

`sexmale` is the shift: males are predicted to be that many grams heavier than females **at the same flipper length**. Geometrically this is **two parallel lines** (same slope, different intercepts):

$$\widehat{\text{mass}} = b_0 + b_{\text{male}} \cdot (\text{is male}) + b_1 \cdot \text{flipper}.$$

## 9.2 Predicting a value from the equation (by hand)

Suppose the fitted model is $\widehat{\text{mass}} = b_0 + b_{\text{male}} \cdot \mathbb{1}[\text{male}] + b_1 \cdot \text{flipper}$. For a **male** penguin with a 195 mm flipper, plug in `is male = 1`:

```r
b <- coef(m3)
b["(Intercept)"] + b["sexmale"] * 1 + b["flipper_length_mm"] * 195
```

```
(Intercept)
   4085.732
```

For a **female** with the same flipper, use `is male = 0` (drop that term). `predict()` does the same arithmetic for you:

```r
predict(m3, newdata = data.frame(sex = c("male", "female"),
                                 flipper_length_mm = c(195, 195)))
```

```
       1        2
4085.732 3486.388
```

> **Exam move**
>
> "Write the equation, then interpret a coefficient" is a classic. Numeric predictor → *"a one-unit increase in $x$ is associated with a $b$ change in $y$, holding the others fixed."* Dummy → *"this level is $b$ higher/lower than the reference level, holding the others fixed."*

# 10 Probability Foundations

Probability is the mathematics of randomness. An **experiment** has an **outcome space** $\Omega$ (all possible outcomes); an **event** $A$ is a subset of $\Omega$; $P(A)$ is a number in $[0, 1]$. Under the **frequentist** view, $P(A)$ is the long-run fraction of trials in which $A$ happens.

## 10.1 The core rules

- **Complement:** $P(A^c) = 1 - P(A)$. *(Great for "at least one ..." — take 1 minus "none".)*
- **Addition / inclusion–exclusion:**

$$P(A \cup B) = P(A) + P(B) - P(A \cap B).$$

  If $A, B$ are **mutually exclusive** (can't both happen), the last term is 0.
- **Multiplication (independence):** if $A, B$ are **independent**,

$$P(A \cap B) = P(A)\,P(B).$$

- **Conditional probability:**
$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}.$$
If $A, B$ are independent then $P(A \mid B) = P(A)$ (knowing $B$ tells you nothing).

## 10.2 Counting (for equally-likely outcomes)

When every outcome is equally likely, $P(\text{event}) = \dfrac{\#\text{ outcomes in event}}{\#\text{ total outcomes}}$.

| Tool | Meaning | R |
|---|---|---|
| $n^k$ | $k$ picks *with* replacement, order matters | `n^k` |
| $n!$ | orderings of $n$ items | `factorial(n)` |
| $\binom{n}{k} = \dfrac{n!}{k!(n-k)!}$ | choose $k$ of $n$, order **doesn't** matter | `choose(n, k)` |

```
# P(exactly 4 red bowties in 10 days, 3 colors) = C(10,4) * 2^6 / 3^10
choose(10, 4) * 2^6 / 3^10
```

```
[1] 0.2276076
```

## 10.3 Some named distributions

Each row lists the **pmf** $P(X = k)$ — the probability the variable lands on a particular value $k$. (For the geometric, $k = 1, 2, \ldots$ is the trial on which the first success occurs.)

| Distribution | Situation | Params | pmf $P(X = k)$ | Mean |
|---|---|---|---|---|
| **Bernoulli** | one 0/1 trial | $p$ | $P(X = 1) = p, P(X = 0) = 1 = p$ | $p$ |
| **Binomial** | # successes in $n$ independent trials | $n, p$ | $\binom{n}{k} p^k (1-p)^{n-k}$ | $np$ |
| **Geometric** | # trials until first success | $p$ | $(1-p)^{k-1} p$ | $1/p$ |

| Distribution | Situation | Params | pmf $P(X=k)$ | Mean |
|---|---|---|---|---|
| **Hypergeometric** | # successes, drawn *without* replacement | $N, G, n$ | $\binom{G}{k}\binom{N-G}{n-k}/\binom{N}{n}$ | $nG/N$ |
| **Poisson** | # rare events in a fixed interval | $\lambda$ | $e^{-\lambda}\lambda^k/k!$ | $\lambda$ |
| **Uniform (discrete)** | equally-likely values $1, \ldots, n$ | $n$ | $1/n$ | $(n+1)/2$ |

The **binomial** is the workhorse. Its pmf,

$$P(X=k) = \binom{n}{k} p^k (1-p)^{n-k},$$

has two pieces: $p^k(1-p)^{n-k}$ is the chance of **one specific** sequence with $k$ successes, and $\binom{n}{k}$ counts **how many** such sequences exist. That split is the whole idea behind the next section.

## 10.4 Order vs. count: when do I need $\binom{n}{k}$?

The binomial coefficient is a **counter of orderings**. Whether it belongs in your answer comes down to one question: does the problem pin down *which* trials turn out a certain way, or only *how many*?

Roll a fair die 5 times. Call a roll **low** if it shows less than 3 (a 1 or 2), so $P(\text{low}) = 2/6 = 1/3$.

**"Exactly 3 of the 5 rolls are low" (any order).** You don't care *which* three rolls are the low ones, so you count the ways to place them, $\binom{5}{3}$, and every such arrangement has the same probability $(1/3)^3(2/3)^2$ (three lows and two non-lows, where "non-low" is the complement with probability 2/3):

$$P = \binom{5}{3}\left(\tfrac{1}{3}\right)^3\left(\tfrac{2}{3}\right)^2 = 10 \cdot \tfrac{1}{27} \cdot \tfrac{4}{9} = \tfrac{40}{243} \approx 0.165.$$

This is exactly the Binomial(5, 1/3) pmf at $k = 3$.

**"The first 3 rolls are low, the last 2 are more than 3" (a fixed order).** Every position is specified, so there is only **one** arrangement and therefore **no** $\binom{5}{3}$ — you just multiply the per-roll probabilities. Read the events carefully: "more than 3" is $\{4, 5, 6\}$, which is **not** the complement of "less than 3" (the value 3 belongs to neither), so $P(\text{more than } 3) = 3/6 = 1/2$:

$$P = \left(\tfrac{1}{3}\right)^3\left(\tfrac{1}{2}\right)^2 = \tfrac{1}{108} \approx 0.0093.$$

The contrast is the entire lesson. Fixing the *count* (any order) brings in $\binom{n}{k}$; fixing the *exact sequence* does not. In fact, if the last two rolls had been required to be "not low" (probability 2/3), the fixed-order answer would be $(1/3)^3(2/3)^2$ — a single term — and the any-order answer is just $\binom{5}{3}$ copies of that one term.

```
# Any order: exactly 3 of 5 low  → binomial, coefficient included
choose(5, 3) * (1/3)^3 * (2/3)^2
```

```
[1] 0.1646091
```

```
dbinom(3, size = 5, prob = 1/3)          # identical
```

```
[1] 0.1646091
```

```
# Fixed order: first 3 low, last 2 more than 3 (prob 1/2 each) → no coefficient
(1/3)^3 * (1/2)^2
```

```
[1] 0.009259259
```

## 10.5 Random variables: expectation & variance

A **random variable** $X$ assigns a number to each outcome. Its **pmf** is $f(x) = P(X = x)$.

$$E(X) = \sum_x x\, f(x) \qquad \mathrm{Var}(X) = E(X^2) - \big(E(X)\big)^2 \qquad SD(X) = \sqrt{\mathrm{Var}(X)}$$

$E(X)$ is a probability-weighted average — the "balancing point" of the distribution.

```
# By hand for a weighted die: E(X) = sum(x * f(x)), Var(X) = E(X^2) - E(X)^2
x  <- c(1, 2, 3, 4, 5, 6)
fx <- c(0.1, 0.3, 0.1, 0.2, 0.1, 0.2)   # probabilities, sum to 1
sum(x * fx)                              # E(X)
```

```
[1] 3.5
```

```
sum(x^2 * fx) - (sum(x * fx))^2          # Var(X)
```

```
[1] 2.85
```

### 10.5.1 Scaling and adding random variables

Real problems rarely ask about $X$ alone — they scale it, shift it, or combine it with another variable. Two rules cover almost everything.

**Expectation is linear — always** (no independence required):

$$E(aX+b) = aE(X)+b, \qquad E(aX+bY) = aE(X)+bE(Y).$$

**Variance squares the coefficients, ignores shifts, and — for *independent* variables — adds:**

$$\mathrm{Var}(aX+b) = a^2\,\mathrm{Var}(X), \qquad \mathrm{Var}(aX+bY) = a^2\,\mathrm{Var}(X)+b^2\,\mathrm{Var}(Y).$$

Two facts baked into that second line trip people up constantly:

- A constant shift $b$ moves the center but **not** the spread, so it drops out of the variance entirely: $\mathrm{Var}(2X-1) = 2^2\,\mathrm{Var}(X)$.
- Because coefficients come out **squared**, variances still **add** for a *difference*: $\mathrm{Var}(X-Y) = \mathrm{Var}(X)+\mathrm{Var}(Y)$ for independent $X, Y$. Subtracting variables makes the result *more* variable, not less.

**Worked examples.** Let $X$ and $Y$ be independent with $E(X) = 2$, $\mathrm{Var}(X) = 3$ and $E(Y) = 5$, $\mathrm{Var}(Y) = 4$.

- *Mean of* $3X+4Y$*:* $E(3X+4Y) = 3E(X)+4E(Y) = 3(2)+4(5) = 26$.
- *Variance of* $3X+4Y$*:* $3^2\,\mathrm{Var}(X)+4^2\,\mathrm{Var}(Y) = 9(3)+16(4) = 91$.
- *Variance of* $2X-1$*:* $2^2\,\mathrm{Var}(X) = 4(3) = 12$ — the $-1$ contributes nothing.
- *Variance of* $2X-Y$*:* $2^2\,\mathrm{Var}(X)+(-1)^2\,\mathrm{Var}(Y) = 4(3)+4 = 16$ (still a sum).

Special cases worth memorizing: Binomial has $E = np$ and $\mathrm{Var} = np(1-p)$; Poisson has $E = \mathrm{Var} = \lambda$. A Binomial$(n,p)$ is literally a sum of $n$ independent Bernoulli$(p)$ variables, which is *why* its mean and variance are just $n$ times $p$ and $p(1-p)$.

> **Exam move**
>
> Build two reflexes: (1) an added or subtracted **constant** never touches a variance; (2) coefficients leave a variance **squared** and always with a + sign, even when the variables are subtracted. If you ever find yourself writing $\mathrm{Var}(X-Y) = \mathrm{Var}(X)-\mathrm{Var}(Y)$, stop — a variance can never be negative.

## 10.6 The CDF, and converting to/from the pmf

The **cumulative distribution function (CDF)** of $X$ is

$$F(x) = P(X \leq x) = \sum_{t \leq x} f(t).$$

For a discrete variable $F$ is a **right-continuous step function**: flat between the possible values, jumping up at each one. Travelling in either direction is mechanical:

- **pmf → CDF:** the CDF at $x$ is the **running total** of the pmf up to and including $x$ (a `cumsum`).
- **CDF → pmf:** the pmf at $x$ is the **size of the jump** in $F$ at $x$, i.e. $f(x) = F(x) - F(x^-)$ — the current level minus the previous one.

Everything left of the smallest value has $F = 0$; everything right of the largest has $F = 1$.

### 10.6.1 pmf → CDF (build the steps)

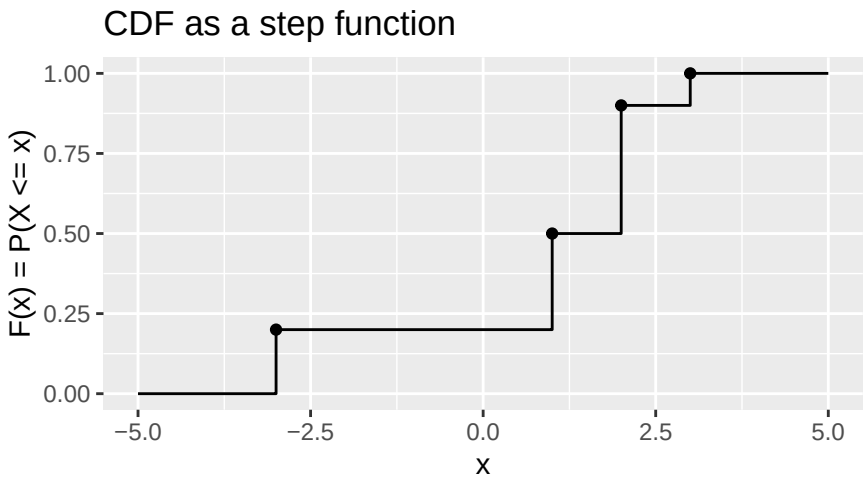
Take $X$ on the support $\{-3, 1, 2, 3\}$ with pmf $0.2, 0.3, 0.4, 0.1$. Accumulate to get the CDF:

| $x$ | $f(x) = P(X = x)$ | $F(x) = P(X \leq x)$ |
|---|---|---|
| $-3$ | 0.2 | 0.2 |
| 1 | 0.3 | 0.5 |
| 2 | 0.4 | 0.9 |
| 3 | 0.1 | 1.0 |

```
d <- tibble(x = c(-3, 1, 2, 3),
            f = c(0.2, 0.3, 0.4, 0.1)) |>
  mutate(F = cumsum(f))        # pmf → CDF is just a running total
d
```

```
# A tibble: 4 x 3
      x     f     F
  <dbl> <dbl> <dbl>
1    -3   0.2   0.2
2     1   0.3   0.5
3     2   0.4   0.9
4     3   0.1   1
```

```r
# CDF step plot: flat, then a jump of size f(x) at each value
steps <- tibble(x = c(-5, -3, 1, 2, 3, 5),
                F = c(0, 0.2, 0.5, 0.9, 1, 1))
ggplot(steps, aes(x, F)) +
  geom_step(direction = "hv") +
  geom_point(data = d, aes(x, F)) +          # closed dot = value is included
  scale_y_continuous(limits = c(0, 1)) +
  labs(title = "CDF as a step function", y = "F(x) = P(X <= x)")
```




### 10.6.2 CDF → pmf (read the jumps)

Now reverse it. Suppose you are *handed* that step plot — a CDF sitting at heights 0, 0.2, 0.5, 0.9, 1.0 with jumps at $-3, 1, 2, 3$. The pmf is exactly the jump sizes, i.e. the successive differences of the levels:

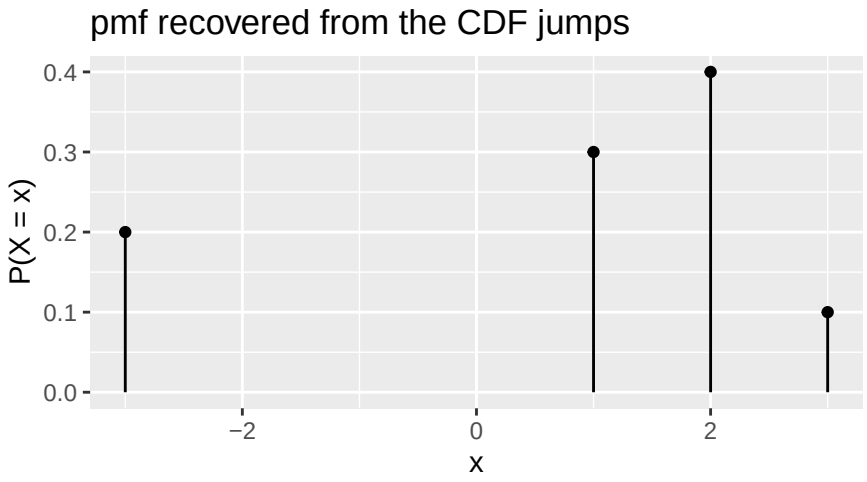
$$f(-3) = 0.2-0 = 0.2, \quad f(1) = 0.5-0.2 = 0.3, \quad f(2) = 0.9-0.5 = 0.4, \quad f(3) = 1.0-0.9 = 0.1.$$

```r
# recover the pmf from the CDF levels with diff()
levels_F <- c(0, 0.2, 0.5, 0.9, 1.0)   # F just below -3, then at -3, 1, 2, 3
xvals    <- c(-3, 1, 2, 3)
pmf      <- diff(levels_F)             # jump sizes = P(X = x)
tibble(x = xvals, pmf)
```

```
# A tibble: 4 x 2
      x   pmf
  <dbl> <dbl>
1    -3   0.2
```

```
2     1   0.3
3     2   0.4
4     3   0.1
```

```
ggplot(tibble(x = xvals, pmf), aes(x, pmf)) +
  geom_segment(aes(xend = x, yend = 0)) +   # a "needle" at each value
  geom_point() +
  labs(title = "pmf recovered from the CDF jumps", y = "P(X = x)")
```




The two pictures hold the same information: a bar of height $f(x)$ in the pmf is the same number as a step of height $f(x)$ in the CDF.

## 10.7 The `d` / `p` / `r` functions

For each named distribution R gives three functions (`binom`, `pois`, `hyper`, ...):

- `d...(x, ...)` — **pmf**: $P(X = x)$
- `p...(q, ...)` — **cdf**: $P(X \le q)$
- `r...(n, ...)` — **random draws**

```
dbinom(x = 3, size = 10, prob = 0.4)   # P(X = 3),  X ~ Binomial(10, 0.4)
```

```
[1] 0.2149908
```

```
pbinom(q = 3, size = 10, prob = 0.4)   # P(X <= 3)
```

```
[1] 0.3822806
```

```r
dpois(x = 2, lambda = 1)                # P(X = 2),  X ~ Poisson(1)
```

```
[1] 0.1839397
```

## 10.8 Random number generation in R

Simulation lets you *check* a probability you worked out by hand, or *estimate* one that's too messy to compute. Three tools do almost all of it. Put `set.seed()` before any of them and the draws become **reproducible** — you get the same "random" numbers every run, which is what you want on homework and exams.

### 10.8.1 1. `sample()` — draw from a vector

`sample(x, size, replace)` pulls `size` elements out of the vector `x`. Set `replace = TRUE` to draw *with* replacement (the box is refilled after each draw) and `replace = FALSE` (the default) to draw *without* (each element used at most once). This is the ticket-box idea in code.

```r
set.seed(1)
box <- c(rep("red", 45), rep("blue", 25))    # 70 tickets: 45 red, 25 blue

sample(box, 5, replace = TRUE)               # 5 draws WITH replacement
```

```
[1] "blue" "red"  "red"  "red"  "red"
```

```r
sample(1:6, 3, replace = TRUE)               # roll a fair die 3 times
```

```
[1] 6 2 3
```

```r
sample(1:6, 6, replace = FALSE)              # a random ordering, no repeats
```

```
[1] 3 1 5 4 2 6
```

A single draw isn't an experiment yet — you almost always **summarize** it. Drawing 10 tickets and counting the reds is one trial of a box experiment:

```r
set.seed(1)
sum(sample(box, 10, replace = FALSE) == "red")   # reds in one draw of 10
```

```
[1] 7
```

### 10.8.2 2. `replicate()` — repeat an experiment many times

`replicate(n, expr)` runs `expr` `n` times and collects the answers. Wrap a `sample()`-plus-summary inside it to build up an **empirical distribution**; the frequentist view says the simulated proportion closes in on the true probability as `n` grows.

```
# one experiment: draw 10 tickets, is the number of reds exactly 3?
set.seed(1)
sum(sample(box, 10, replace = FALSE) == "red") == 3   # TRUE / FALSE
```

```
[1] FALSE
```

```
# repeat that experiment 10,000 times, then take the proportion that were TRUE
set.seed(1)
got3 <- replicate(10000, sum(sample(box, 10, replace = FALSE) == "red") == 3)
mean(got3)                                             # estimated P(exactly 3 red)
```

```
[1] 0.0194
```

(Start with 100 reps to see the idea; more reps just tighten the estimate.) The same pattern estimates any event — here the classic "exactly 3 heads in 5 flips," whose true value is $\binom{5}{3}/2^5 = 0.3125$:

```
set.seed(1)
coin   <- c(0, 1)                                     # 0 = tails, 1 = heads
trials <- replicate(1000, sum(sample(coin, 5, replace = TRUE)))
mean(trials == 3)                                      # ≈ 0.3125
```

```
[1] 0.315
```

### 10.8.3 3. `rnorm()`, `rbinom()`, ... — draw straight from a named distribution

When the experiment *is* a named distribution, skip the `sample()` scaffolding and draw from it directly. Every distribution has an `r...` function whose **first argument is how many draws you want**, followed by the distribution's parameters.

```
set.seed(1)
rbinom(n = 5, size = 10, prob = 0.4)   # 5 draws from Binomial(10, 0.4)
```

```
[1] 3 3 4 6 3
```

```
rpois(n = 5, lambda = 2)                # 5 draws from Poisson(2)
```

```
[1] 4 4 2 2 0
```

```
rnorm(n = 5, mean = 100, sd = 15)       # 5 draws from Normal(mean 100, sd 15)
```

```
[1]  87.69297 107.31144 111.07487 108.63672  95.41917
```

```
runif(n = 5, min = 0, max = 1)          # 5 draws from Uniform(0, 1)
```

```
[1] 0.9347052 0.2121425 0.6516738 0.1255551 0.2672207
```

`rbinom(n, size, prob)` is exactly "count the successes in `size` independent trials, and do that `n` times" — the same thing as `replicate(n, sum(sample(...)))` with replacement, just faster and built in. Summarizing a large batch of draws recovers the distribution's mean and variance:

```
set.seed(1)
draws <- rbinom(n = 10000, size = 10, prob = 0.4)
c(mean = mean(draws), var = var(draws))    # near np = 4 and np(1-p) = 2.4
```

```
    mean      var
4.003800 2.452631
```

**`d` / `p` / `r`, side by side**

For any named distribution, `d...` gives the pmf $P(X = k)$, `p...` gives the cdf $P(X \leq q)$, and `r...` gives random draws. So `dbinom` **computes** a probability while `rbinom` **simulates** one — and the average of many `rbinom` draws should land right on top of the exact `dbinom`/`pbinom` answer.

## 10.9 With vs. without replacement: a box of tickets

Many "how many successes did I get?" problems are really about **drawing tickets from a box**, and the single most important question is whether you **replace** each ticket before the next draw.

Put 70 tickets in a box: **45 red** and **25 blue**. Draw 10 of them and count the reds.

- **With replacement** — you look at each ticket, then put it back. Every draw faces the same box, so $P(\text{red}) = 45/70$ stays constant and the draws are **independent**. The red count is **Binomial**$(n = 10,\ p = 45/70)$.
- **Without replacement** — you keep each ticket you draw. The box now changes as you go (draw a red and there is one fewer red left), so the draws are **dependent**. The red count is **Hypergeometric**$(N = 70,\ G = 45,\ n = 10)$.

That one choice — replace or not — *is* the binomial-vs-hypergeometric fork. With replacement the trials are independent (binomial); without, they are linked through the shrinking box (hypergeometric).

```
box <- c(rep("red", 45), rep("blue", 25))   # the 70-ticket box

set.seed(1)
# WITH replacement → simulate, then compare to the exact Binomial pmf
wr <- replicate(10000, sum(sample(box, 10, replace = TRUE) == "red"))
c(sim = mean(wr == 7), exact = dbinom(7, size = 10, prob = 45/70))
```

```
      sim     exact
0.2418000 0.2480324
```

```
# WITHOUT replacement → simulate, then compare to the exact Hypergeometric pmf
wor <- replicate(10000, sum(sample(box, 10, replace = FALSE) == "red"))
c(sim = mean(wor == 7), exact = dhyper(7, m = 45, n = 25, k = 10))
```

```
      sim     exact
0.2656000 0.2631004
```

In R's `dhyper(x, m, n, k)`, `m` is the number of "successes" in the box (reds, 45), `n` the number of non-successes (blues, 25), and `k` the number drawn (10). The two exact probabilities come out close but not equal — sampling without replacement makes extreme counts slightly less likely, because each draw nudges the box against repeating itself.

---